

Claude Desktop Vulnerability: One-Click Prompt Injection

Claude Desktop Vulnerability: One-Click Prompt Injection - Elad Luz, Oasis Security.

- Published: 2026-07-28
- Original: <<https://www.oasis.security/blog/claude-desktop-vulnerability>>
- Preserved from: <https://www.oasis.security/blog/claude-desktop-vulnerability> (live) on 2026-09-18
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Claude Desktop Vulnerability: One-Click Prompt Injection | Oasis

*TL;DR: PromptFiction was a Claude Desktop vulnerability: a single click on a crafted link could hand instructions to the AI agent and make it act on them, with no send button and no chance for the user to read them first. Depending on the configuration, that ranged from quietly copying someone's private conversations to running an attacker's code. We reported it to Anthropic, who fixed it, so updating Claude Desktop protects you. The lesson outlasts the bug, and the full technical breakdown is [**here](#).*

Most people think of an AI assistant as something they talk to. You type, it answers. The safety of that exchange rests on one quiet assumption: the instructions came only from you.

We found a case where that assumption did not hold. We call the flaw PromptFiction. It is a Claude Desktop vulnerability that turns a single link into an instruction the app obeys.

We have looked at this theme before. In earlier research, [Three Vulnerabilities, One Click](#), a hidden prompt could be loaded into a Claude chat via a link. That attack still needed one thing from the victim: pressing send. PromptFiction removes even that. The click is the whole attack.

How did the Claude Desktop vulnerability work?

[Claude Desktop](#), Anthropic's app for your computer, can be opened via a link, just like a Zoom or email link opens its app. But a crafted version of that link could do more than open the app: it could carry a full set of instructions and prompt the assistant to act on

them immediately. No send button. No time to read the text and decide. One click was the whole interaction.

The instructions could also be hidden in plain sight. The visible part could be something harmless, like a request to help you plan a trip, with the real instructions folded out of view below it. The user saw a friendly opening. The assistant received the rest. See exactly how the link worked in the [technical write-up](#).

Why does the Claude Desktop vulnerability matter?

What an attacker could do came down to what the assistant could already reach.

On a standard install, that included the user's own conversation history: source code and internal documents, unreleased plans, customer data, and details of how systems are built and secured. A single click could tell the assistant to gather it and send it out. On machines set up for coding, where the assistant can read and write files, the same click could go further, to the point of running the attacker's code.

We will be precise about the tone. This is not a story about AI turning against anyone. The assistant did what it is built to do: follow the prompt in front of it. The failure was upstream: the system let a prompt reach the agent with no proof that a human wrote it or agreed to send it.

Is Claude Desktop safe now?

We reported PromptFiction to Anthropic through their Responsible Disclosure Program, and they fixed it. Claude Desktop no longer acts on instructions delivered this way on its own. A prompt that arrives through one of these links now waits, pre-filled, for the user to read it and press send. The human is back in the loop. If you use Claude Desktop, update to the latest version 1.1.2321 or later.

How to protect against prompt injection

For most people, the takeaway is simple. Keep your AI apps updated, and treat a link that opens an app with the same caution you give a link that downloads a file.

For any company deploying AI agents, it is bigger. As you give agents more access, ask three questions about each one:

- Which instructions can reach this agent, and from where?
- Can we tell the instruction the user intended from one that arrived another way?
- Before the agent takes a real action, does anything confirm that a human approved this specific step?

This is the work we do at [Oasis Security](#). AI agents and their [related non-human identities need governance at the moment of action](#), not just at onboarding. An agent should earn access to one task at a time, based on the intent behind the request, and lose it when the task is done. Standing access and blanket trust are what turn a single click into a breach.

Keep Learning

Building The Next-Generation AI Security Platform Danny Brickman • July 28, 2026

Claude Tag: Agent Identity and the NHI Governance Gap Marta Dern • July 1, 2026

Extending Zero Trust to Non-Human & Agentic Identities Leandro Iacono • June 10, 2026

AI Access Management

Archived from <https://www.oasis.security/blog/claude-desktop-vulnerability>. Rendered offline from the local Markdown copy.